

第4章. Agent 与工具：从函数到资料检索助手

怎样把项目函数变成模型可以选择的能力？

学习目标

1. 理解工具名、描述、Schema 与参数校验。
2. 区分模型提出调用与程序执行函数。
3. 阅读资料检索工具与按 ID 查询工具。
4. 认识独立 Agent 入口及它与 Web 工作流的分工。

阅读方法：先看业务问题和图，再读名词与源码，最后复制完整实验验证。代码节选用于定位实现；完整实验会明确运行条件。

代码基准：2026-10-03 本地项目当前源码。默认配置与环境覆盖值可能不同；本文不以旧实施计划代替真实实现。

运行约定：命令从项目根目录执行。学习实验代码直接收录在 PDF 内；自行新建练习文件后复制运行，不另附脚本。服务实验的结果应由你实际运行记录。

学习产出：每章至少留下一份自己的输入、输出和解释。涉及检索质量、耗时或简历中的改善幅度，只采用有证据的数据。

本章目录

4.1 需求分析：为什么仅有聊天模型还不够 3

4.2 名词解析：Agent、Tool 与 Tool Call 3

4.3 工具输入：先看 Schema，再看装饰器 5

4.4 读懂两个真实工具的内部流程 6

4.5 Agent 创建：模型、工具、提示词各负责什么 9

4.6 Agentic RAG：理解组合，同时说清能力边界 10

4.7 离线实验：用真实输入与结果验证理解 11

4.8 常见错误、复习题与源码阅读索引 14

学习目标：看懂工具怎样被描述、校验和执行；分清模型的工具决策与程序的真实检索；能够沿着当前项目源码解释一次 Agent 调用。代码与实验以本地实现为准，课程用于提供学习顺序和讲解方式。

4.1 需求分析：为什么仅有聊天模型还不够

4.1.1 从一句用户问题开始

用户输入“帮我找东风天龙仪表电路图”。模型可以读懂这句话，也可能知道东风、天龙和仪表的含义。但它并不知道本地 CSV 此刻有哪些记录，更不能可靠地给出这份资料库里的文档 ID。

这个场景需要两个能力：理解用户在找什么，以及访问真实的资料库。前者由模型和查询处理模块完成，后者由检索代码完成。Agent 把模型与这些可执行能力连接起来，让模型能够在需要外部信息时提出工具调用。

再看另一个问题：“查看 ID 为 1 的资料。”用户已经给出明确标识，最合适的操作是精确查表。此时重新进行语义搜索既绕远路，也可能把其他相似资料排在前面。因此项目提供了搜索工具和按 ID 查看工具，分别覆盖模糊查找与精确查找。

4.1.2 本章需要解决哪些问题

第一，模型应该什么时候调用哪个工具？第二，工具收到的参数是否合规？第三，工具结果如何回到模型？第四，如何要求回答只引用资料库中真实存在的标题与 ID？第五，哪些硬规则还需要交给程序控制？

这些问题决定了本章的实现顺序：先读工具输入 Schema，再读工具函数，再看工具如何注册到 Agent，最后沿着调用入口解释模型与工具之间的循环。先理解数据和职责，随后再阅读框架 API，学习会更连贯。

4.1.3 当前项目的实现边界

当前独立 Agent 使用 LangChain 的 `create_agent()`。它拥有两个资料工具，并通过系统提示约束资料结果的来源。入口是 `scripts/run_agent.py`。

浏览器聊天的当前主流程由自定义 LangGraph 接管。`app/api/chat.py` 调用 `run_search_workflow()` 和 `resume_search_workflow()`，没有直接调用 `run_circuit_agent()`。两种入口共用模型与检索能力，但业务控制方式不同。

资料库当前保存的是 CSV 中的资料标题、ID 和层级路径。按 ID 查看不会打开实际 PDF、读取电路图图像，也不会返回电路图正文。本章学习的是资料导航助手，不应把它描述成已经能够阅读线路细节的维修诊断系统。

4.2 名词解析：Agent、Tool 与 Tool Call

4.2.1 Agent：带有工具决策的执行循环

Agent（智能体） 在本章中的具体含义，是模型根据当前消息决定直接回答，或请求执行某个工具；执行器把工具结果放回消息上下文，再让模型继续判断下一步。

“智能体”这个名称很宽泛。理解当前代码时，不需要预设它必然拥有长期记忆、自我规划、多智能体协作或自动评估。本项目的独立 Agent 首先展示的是模型、工具、提示词和消息循环的组合。

4.2.2 Tool: 让程序能力成为模型可选择的动作

Tool (工具) 是有明确名称、说明、输入结构和执行逻辑的程序能力。模型看到的是“有什么工具、分别能做什么、需要哪些参数”；程序运行时调用的则是对应的 Python 函数。

普通函数已经能执行查询，但模型无法凭空知道某个函数的名称和参数。`@tool` 把函数包装成 LangChain 工具，结合函数名、文档字符串和输入 Schema 提供可供模型使用的描述。

4.2.3 Tool Call: 模型提出请求，程序真正执行

Tool Call (工具调用请求) 通常包括工具名称、参数和调用 ID。例如，模型可能提出调用 `get_document_by_id`，参数是 `{"document_id": 1}`。

模型输出这个请求，并不等于模型自己执行了 Python、读取了 CSV。执行器找到对应工具，完成参数校验，运行函数，再把结果作为工具消息返回。调用 ID 用于把请求与相应的结果关联起来，尤其是在存在多个调用时。

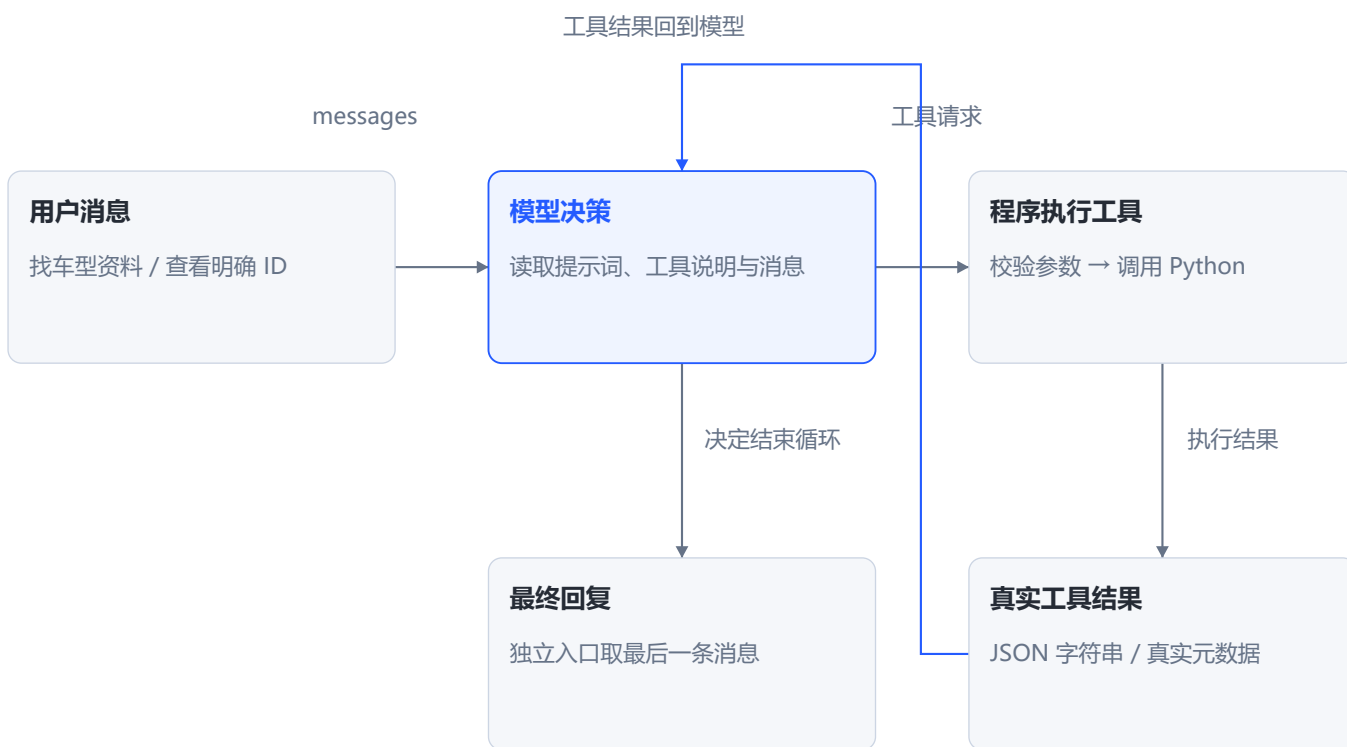


图4-1. 模型提出调用请求，程序执行真实工具；结果回到模型后，可以继续调用工具或组织最终回复。

是否继续调用工具由模型决定，真实数据读取由程序完成；模型最终回答仍需要遵守资料来源约束。

4.2.4 四个概念的职责对照

概念	在当前项目中解决什么	关键位置
模型 Model	理解问题、提出工具请求、组织回复	app/core/model.py
输入 Schema	声明工具参数与数值范围	app/schemas/tool.py
工具 Tool	执行搜索或按 ID 查表	app/tools/
Agent	注册模型与工具、运行消息循环	app/agents/circuit_agent.py
自定义 Graph	执行候选数量阈值、多轮澄清与改选	app/graph/

这里的 Schema 是数据契约，不是知识库结构。它约束调用输入；返回结果另外由工具函数构造。区分输入契约与结果格式，才能正确定位参数错误和展示错误。

4.3 工具输入：先看 Schema，再看装饰器

4.3.1 搜索参数怎样被声明

源码片段，来自 app/schemas/tool.py:

代码块 python

```
1 class SearchKnowledgeBaseInput(BaseModel):
2     query: str = Field(
3         min_length=1,
4         description="保留原始车型和 ECU 编码的检索问题",
5     )
6     top_k: int = Field(
7         default=5,
8         ge=1,
9         le=20,
10        description="最多返回多少条候选文档",
11    )
```

这一小段需要从三个层面理解。query: str 声明字符串输入；min_length=1 阻止空字符串；description 告诉模型应该如何填写参数。top_k 有默认值 5，同时必须位于 1 到 20 之间。

默认值意味着调用者可以只提供查询文本。范围意味着模型即使提出 top_k=100，也不能顺利进入工具业务函数。校验会先拒绝不符合契约的参数，这是程序约束，可靠性与提示词里的“请返回五条”不同。

注意，min_length=1 按字符串长度判断，当前 Schema 本身没有执行去首尾空格的验证器。字符串 "" 长度大于零，可以通过该字段校验。不能把“长度校验”描述成完整的空白文本清洗；阅读代码时要检查清洗发生在哪一层。

4.3.2 精确查看参数怎样被声明

源码片段，同样来自 `app/schemas/tool.py`:

代码块 python

```
1 class GetDocumentInput(BaseModel):
2     document_id: int = Field(
3         gt=0,
4         description="车辆电路图文档 ID",
5     )
```

`gt=0` 要求 ID 大于零。因此 0 与负数属于输入错误，而一个合法但不存在的正整数属于查询未命中。这两类情况应分别处理：前者不应进入数据查找，后者可以正常执行并返回 `not_found`。

Schema 使用 Pydantic 默认行为，没有声明严格类型模式。部分可转换的输入可能被转换为目标类型，例如数字字符串可能成为整数。不要仅根据类型注解就断言所有非整数对象都会被拒绝；如果业务需要严格类型，应另行设计并测试。

4.3.3 @tool 连接契约与执行代码

源码片段，来自 `app/tools/get_document.py`:

代码块 python

```
1 @tool(args_schema=GetDocumentInput)
2 def get_document_by_id(document_id: int) -> str:
3     """根据文档 ID 获取车辆电路图资料的标题和层级路径。"""
4     doc = get_document(document_id)
```

阅读顺序是装饰器、签名、文档字符串、函数主体。装饰器指定输入模型；函数签名说明传入 ID、返回字符串；文档字符串解释用途；主体调用本地资料索引。

被装饰后的对象是工具对象。学习实验可使用 `get_document_by_id.invoke({"document_id": 1})`，这会通过工具接口进入校验与执行过程。不要把装饰后的对象完全当成未包装的普通函数来理解。

描述文字影响模型能否正确选择工具，但不会保证模型每次选择都正确。输入 Schema 能控制参数形状与范围，但也不能保证查询含义准确。二者各有作用，需要配合真实结果约束、业务路由和测试使用。

4.4 读懂两个真实工具的内部流程

4.4.1 search_knowledge_base: 搜索并返回真实候选

该工具位于 `app/tools/search_knowledge_base.py`。它先调用 `get_chat_model()` 获取查询处理所需的模型，再调用 `prepare_search_query()`，得到原始查询、改写查询、结构化意图和过滤条件。

接下来调用 `retrieve_docs()`，把改写文本传给检索链路，同时把过滤条件作为独立参数传入。底层采用 Dense、BM25、RRF 与 Cross-Encoder。检索结果是资料记录，不是模型自行生成的标题。

源码片段，只展示查询准备与检索调用：

代码块 python

```
1 model = get_chat_model()
2 query_plan = prepare_search_query(query, model=model)
3 docs = retrieve_docs(
4     query_plan.rewritten_query,
5     top_k=top_k,
6     filter_query=query_plan.filter_query,
7 )
```

这里容易出现一个误解：以为 Agent 已经调用了聊天模型，所以工具执行时只会访问数据库。实际搜索工具内部还进行结构化意图提取与查询改写，因此一次工具调用可能包含额外模型请求。评估延迟与费用时要计算整条链路。

工具把结果组织为一个 JSON 对象，然后执行 `json.dumps(..., ensure_ascii=False)`。返回类型是字符串，字符串内容才是 JSON。调用者需要解析时应使用 `json.loads()`；不要把返回值直接按 Python 字典下标访问。

4.4.2 搜索工具返回字段怎么读

字段	含义	阅读时关注的问题
original_query	用户原问题	是否保留原始需求
rewritten_query	用于召回的改写文本	型号、数字、缩写是否保留
intent	结构化检索意图	是否正确抽取品牌、车型等
filter_query	检索过滤表达式	是否过于严格导致漏召回
count	实际返回候选数量	不应假定一定等于 top_k
documents	ID、标题、层级路径、score 列表	是否都来自真实记录

`count` 是 `len(docs)`。`top_k=5` 表示最多返回五条，不代表一定找到五条。数据不存在、过滤较严、重排阈值淘汰候选，都可能减少数量。

`score` 用于排序与观察匹配程度。它应结合实际重排器的评分语义解释，不应未经校准就当成“答案正确概率”。分数高也不证明标题对应的文件内容满足用户全部需求。

4.4.3 get_document_by_id：查表并明确区分命中与未命中

按 ID 工具直接调用 `get_document()`。命中时返回 `status="found"`，并给出 `document.id`、`document.title`、`document.hierarchy_path`；未命中时返回 `status="not_found"` 和请求的 `document_id`。

下面是结构示意图，字段来自当前实现；标题值仅用于说明格式，不代表新增资料：

代码块 json

```
1 {
2   "status": "found",
3   "document": {
4     "id": 1,
5     "title": "真实 CSV 记录中的标题",
6     "hierarchy_path": "真实 CSV 记录中的层级路径"
7   }
8 }
```

未命中时的结构没有 document 字段。消费这个结果的代码应先判断 status，再访问命中内容。缺失资料不是系统故障，也不应该补出一个相似标题来伪装命中。

4.4.4 DocumentStore 在哪里，为什么不经过向量库

app/rag/document_store.py 把 CSV 加载结果变成 dict[int, Document]，键是文档 ID。查询时执行字典的 .get(document_id)，找到就返回 Document，找不到就返回 None。

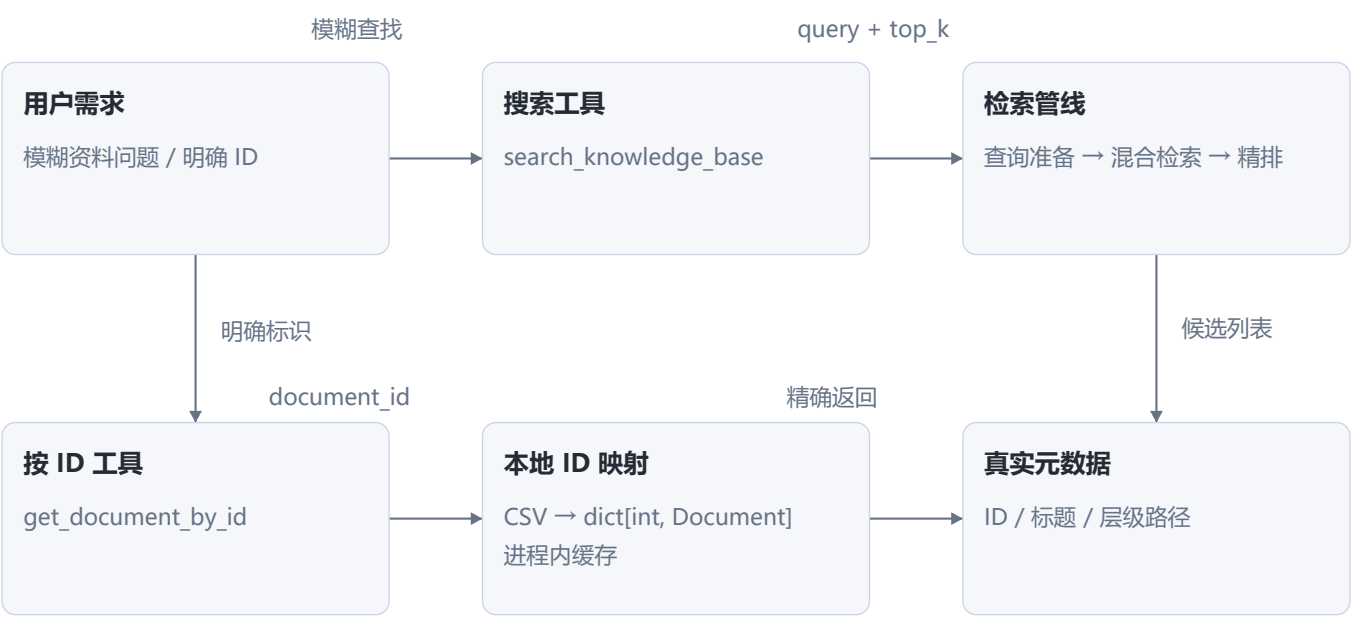


图4-2. 模糊搜索走查询准备与检索；精确 ID 走本地 Document 映射。当前返回资料标题、ID 和层级路径，不读取电路图正文。

模糊搜索走检索管线；明确 ID 走本地映射。两者最终提供的都是当前资料记录中的元数据。

精准标识查询不需要语义相似度，因此不经过 Embedding、Milvus 或 Cross-Encoder。这个设计既直接，也避免相似搜索偏离指定 ID。学习其他项目时可以沿用这种职责判断：用户已经有唯一标识，就优先考虑精确查询。

4.4.5 缓存到底缓存了什么

`get_document_index()` 使用 `@lru_cache`。函数没有参数，首次调用构建整份 ID 映射，后续调用复用同一进程内的索引。它缓存的是 Document 映射，不是大模型回答，也不是查询结果列表。

CSV 在进程运行时发生修改，旧映射不会自动刷新。需要清理该函数缓存或重新启动进程后才能重新构建。清理前应考虑调用方是否还持有旧对象；学习实验不必修改数据文件。

`get_circuit_agent()` 也使用 `@lru_cache`，用于复用 Agent 实例。实例缓存与会话记忆不同。当前独立 Agent 的构造没有传入 `checkpointer`，包装调用也只发送本次用户消息，所以不能仅凭“用了缓存”就宣称它具有完整的多轮记忆。

4.5 Agent 创建：模型、工具、提示词各负责什么

4.5.1 先读工具注册，再读提示词

源码片段，来自 `app/agents/circuit_agent.py`：

代码块 python

```
1 model = get_chat_model()
2 tools = [search_knowledge_base, get_document_by_id]
3 agent = create_agent(
4     model=model,
5     tools=tools,
6     system_prompt=SYSTEM_PROMPT,
7 )
```

三个参数提供三类信息。`model` 提供语言理解与工具决策能力，`tools` 声明允许执行的程序能力，`system_prompt` 说明角色和使用规则。只有注册进列表的工具，才属于这个 Agent 可选择的动作集合。

将能力封装为工具之后，业务调用者只需发送消息，不必在入口处手写“出现某关键词就查库”的判断。这适合自然语言输入较多、动作种类明确的场景。不过，当规则必须严格执行时，仍需要程序层校验与路由。

4.5.2 当前系统提示词的学习重点

提示词把助手定位为车辆电路图资料导航助手，并强调：普通问候可以直接回答；查找具体车型、ECU、文档等需求应调用搜索工具；明确 ID 的需求可调用按 ID 工具；只能展示工具真实返回的资料。

提示词还明确说明，多轮候选收敛与“结果超过五条必须继续澄清”的硬规则由后续 LangGraph 负责。它没有把所有业务逻辑都塞进 Agent 提示词，而是为不同控制层保留职责。

这里的“必须调用”表达模型行为要求，并非每次调用前都会由独立 Agent 的外层代码做确定性验证。若要评价真实遵循率，需要使用覆盖不同问法的模型测试；本次离线实验只验证程序包装与工具契约。

4.5.3 一次 Agent 调用怎样进入消息循环

`run_circuit_agent()` 先对输入去首尾空格，空输入直接返回空字符串；然后获取注入的 Agent 或默认缓存 Agent；再使用 `ainvoke()` 发送 messages 输入。

源码片段，来自当前包装函数：

代码块 python

```
1 result = await agent.ainvoke({
2     "messages": [
3         {"role": "user", "content": message}
4     ]
5 })
6 messages = result.get("messages", [])
7 if not messages:
8     return ""
9 return _message_content(messages[-1]).strip()
```

输入不是一个裸字符串，而是带角色的消息列表。框架内部可能经过模型消息、工具请求、工具结果、后续模型消息。包装函数只取最后一条消息的文本，作为命令行输出。

如果最后一条消息内容不是字符串，`_message_content()` 将其转换为字符串。这是当前包装层的简化行为，没有专门提取图片、文件或结构化内容。扩展多模态时需要重新审视输出格式。

4.5.4 独立入口怎么定位

`scripts/run_agent.py` 使用 `argparse` 接收一个位置参数，再用 `asyncio.run()` 进入异步调用。未来接入其他终端界面时，可以复用 `run_circuit_agent()`，无需重新创建工具与提示词。

下面是入口命令，仅用于认识真实入口；它会触发模型，搜索需求还依赖检索服务，本次没有运行：

代码块 powershell

```
1 .\.\venv\Scripts\python.exe -m scripts.run_agent "查看资料 ID 1"
```

能够读懂命令入口，不等于需要立即联网调用。先完成本章离线实验，确认参数、返回结构和包装函数行为，再进行完整模型实验，可以更容易区分代码问题与配置问题。

4.6 Agentic RAG：理解组合，同时说清能力边界

4.6.1 RAG 怎样进入 Agent

RAG (Retrieval-Augmented Generation, 检索增强生成) 强调回答前获得外部资料。**Agentic RAG**进一步让智能体参与决定何时检索、调用何种检索能力，以及是否继续执行下一步动作。

当前独立 Agent 可以选择搜索工具或按 ID 查看工具，再根据结果组织回答。这个组合具备模型驱动的工具决策，但不能因此推导出它已经实现复杂查询规划、自动质量评分、多 Agent 协作或自我纠错检索。

工具内部的查询准备、混合召回与重排属于预先设计的检索流程。模型选择工具和工具内部执行固定流程，是两层不同的控制。在学习时分别画出来，就能解释哪些步骤由模型决定，哪些步骤由代码确定。

4.6.2 为什么 Web 主流程选择自定义 Graph

Web 用户需要看到真实选择题，点击后缩小候选，结果不满意时还能改选或返回。候选大于默认五条时继续澄清，不能靠模型临时决定是否遵守。

因此当前 Web 主流程把查询理解、改写、检索、Facet 构建、暂停选择与最终展示拆成图节点。它仍会使用模型，但节点顺序与数量分支由代码控制。下一章会详细解释这些节点与 checkpoint。

学习维度	独立 Agent 入口	当前 Web Graph 入口
动作选择	模型依据工具描述与提示词提出请求	条件边与节点代码控制业务步骤
结果形式	最后一条消息文本	text、options、result 三类响应
候选收敛	提示词指出交给 Graph	真实候选分组与数量路由
状态恢复	当前包装没有传入会话 checkpoint	sessionId 对应 thread_id
典型阅读入口	app/agents/circuit_agent.py	app/graph/workflow.py

4.6.3 学课程时如何对应项目

飞书 Agent 入门实战先做需求分析，再拆模型、工具、记忆与提示词，最后把流程跑通。本章沿用这种学习顺序，把菜谱搜索替换成车辆资料查找；没有把课程案例的上传图片、SQLite 记忆或网络搜索能力照搬成项目事实。

Runtime、Middleware、MCP 与 Multi-Agent 可以作为后续扩展方向，但当前独立 Agent 没有提供对应的自定义实现。学习这些课程时先说明需求，再判断是否值得引入，避免为了覆盖课程标题给项目虚构功能。

例如，MCP 的价值在于统一外部工具接口；当资料检索需要供多个客户端复用时值得研究。Middleware 可用于统一调用前后处理；当需要审计、预算或访问控制时再考虑。多 Agent 更适合职责确实独立的协作任务，应先证明单条检索流程的质量。

4.7 离线实验：用真实输入与结果验证理解

4.7.1 运行方式与实验范围

以下 Python 代码都可以在项目根目录自行新建临时文件后执行。使用已有 .venv，例如。
`\.venv\Scripts\python.exe your_demo.py`。完整实验不含省略号，不需要下载模型、启动服务或调用付费 API。
离线实验前，在当前 PowerShell 窗口关闭可选追踪，避免已有环境设置启动后台上传线程。以下设置只影响当前终端进程及其子进程，不修改项目配置文件：

代码块 powershell

```
1 $env:LANGSMITH_TRACING = "false"
2 $env:LANGCHAIN_TRACING_V2 = "false"
```

“源码片段”用于阅读实现，不能单独当完整程序运行；“可独立复制运行”包含所需导入与执行代码。本章把两类代码明确标注，避免复制后才发现缺少上下文。

4.7.2 实验一：观察合法参数与非法参数

可独立复制运行：

代码块 python

```
1 from pydantic import ValidationError
2 from app.schemas.tool import SearchKnowledgeBaseInput
3 from app.schemas.tool import GetDocumentInput
4
5 print(SearchKnowledgeBaseInput(query="东风仪表").top_k)
6 for value in [0, 21]:
7     try:
8         SearchKnowledgeBaseInput(query="东风", top_k=value)
9     except ValidationError:
10         print("top_k rejected:", value)
11 try:
12     GetDocumentInput(document_id=0)
13 except ValidationError:
14     print("id rejected: 0")
15 print("spaces length:", len(
16     SearchKnowledgeBaseInput(query=" ").query
17 ))
```

预期并在本地实际观察到的输出：默认值为 5；0 与 21 被拒绝；ID 0 被拒绝；空格字符串长度为 3。最后一项用来检查校验边界，说明业务清洗不能只依赖最小长度。

实验之后可以追问：如果产品明确不允许空白查询，应把去空白逻辑放在输入层、工具层还是公共查询模块？答案应根据调用入口是否共用决定，关键是让所有入口都拥有一致且可测试的契约。

4.7.3 实验二：执行真实的按 ID 工具

可独立复制运行：

代码块 python

```
1 import json
2 from app.rag.document_store import get_document_index
3 from app.tools.get_document import get_document_by_id
4
5 index = get_document_index()
6 first_id = min(index)
7 missing_id = max(index) + 1
8 found = json.loads(get_document_by_id.invoke(
9     {"document_id": first_id}
10 ))
11 missing = json.loads(get_document_by_id.invoke(
12     {"document_id": missing_id}
13 ))
14 print(type(get_document_by_id.invoke(
15     {"document_id": first_id}
16 )).__name__)
17 print(found)
18 print(missing)
19 print("index size:", len(index))
```

本次数据下，实际输出类型为 `str`，索引规模为 4233。首个 ID 为 1，标题是“三一挖掘机_德国仪表显示器针脚定义”，层级为“电路图->ECU电路图->工程机械->三一->德国仪表”。不存在的 ID 返回 `not_found`。

这项实验同时验证三点：工具输出是 JSON 字符串；命中对象只包含当前元数据；不存在的 ID 不会被模糊匹配成其他记录。如果以后 CSV 变更，规模与首条内容应以你的实际输出为准。

4.7.4 实验三：用 FakeAgent 观察包装函数

可独立复制运行：

代码块 python

```
1 import asyncio
2 from langchain_core.messages import AIMessage
3 from app.agents.circuit_agent import run_circuit_agent
4
5 class FakeAgent:
6     async def ainvoke(self, payload):
7         print(payload)
8         return {"messages": [AIMessage(content=" 测试完成  ")]}
9
10 async def main():
11     text = await run_circuit_agent(
12         " 查看资料 ID 1 ", agent=FakeAgent()
13     )
14     print(repr(text))
15     print(repr(await run_circuit_agent(
16         " ", agent=FakeAgent()
17     )))
18
19 asyncio.run(main())
```

实际观察到，传入 FakeAgent 的用户消息已经去掉首尾空格；返回文本为 '测试完成'；纯空格输入返回 ""，且不会执行 FakeAgent。这个实验验证包装层，没有验证真实模型是否会正确发出工具调用。

注入 FakeAgent 的价值在于隔离外部服务。包装函数可以独立检查，网络、模型费用和检索服务都不会干扰结果。真实模型行为仍应由另外的集成实验评价，不能把离线通过解释成完整智能体效果已被证明。

4.7.5 本章相关测试

本次实际执行了 `tests/test_agent.py`、`tests/test_document_store.py`，并结合下一章的路由、Facet、选择与恢复测试检查程序契约。它们覆盖空输入、消息包装、索引字段以及 Graph 的恢复选择行为。

推荐先读断言再读测试构造。断言说明维护者希望稳定保留的行为；FakeAgent 或 FakeGraph 说明哪些外部环节被替换。测试里没有访问模型的行为，不应被理解为对模型决策准确率的评测。

4.8 常见错误、复习题与源码阅读索引

4.8.1 常见错误定位

现象	优先检查	原因与处理思路
工具输入被拒绝	app/schemas/tool.py	ID 或 top_k 越界，先修正参数
JSON 结果无法按键读取	工具返回类型	先 json.loads，再访问字段
指定 ID 查不到	DocumentStore 与 CSV	合法 ID 也可能不存在
CSV 改了却仍返回旧内容	get_document_index 缓存	当前进程复用旧映射
CLI 能跑但 Web 行为不同	两套真实入口	Web 经过自定义 Graph
模型凭空给出标题	工具是否执行、最终消息	需要检查真实调用与来源约束

搜索工具执行失败时，还需区分模型配置、查询处理、Milvus、Embedding 和重排器几个环节。工具层只是组合入口；看到“调用工具失败”不能立刻断定工具 Schema 有问题。

4.8.2 复习题

1. 模型提出 tool call 后，究竟是谁执行 Python 函数？
2. @tool 的文档字符串与 args_schema 分别解决什么问题？
3. 合法但不存在的 ID，与 ID=0 的处理路径有什么差别？
4. get_document_by_id 为什么不需要向量相似度搜索？
5. Agent 实例缓存为什么不等于多轮记忆？
6. 为什么不能直接用独立 Agent 的提示词证明 Web 的五条结果规则？
7. 本章离线实验验证了哪些程序行为，仍未验证哪些模型行为？

4.8.3 参考答案与讲述练习

第一题，执行器调用工具函数，模型只产生调用请求。第二题，说明文字帮助模型理解能力，Schema 控制输入结构和范围。第三题，不存在的正整数进入查表并返回 not_found，0 在输入校验阶段被拒绝。

第四题，明确 ID 是唯一标识查询，字典映射能够直接返回对应记录。第五题，缓存复用实例没有自动保存、读取每个用户会话的消息。第六题，Web 的硬规则存在于 Graph 路由与最终结果节点，需要阅读这些代码。第七题，离线实验验证参数、CSV 索引、返回结构与包装函数，未评价真实模型决策或完整检索质量。

最后用两分钟解释一次资料查询：用户问题进入 Agent，模型根据工具描述提出搜索请求，程序准备查询并执行检索，工具返回真实元数据，模型据此组织最后回复。随后补充当前 Web 使用自定义 Graph 完成多轮澄清，独立 CLI 只展示基础工具智能体。能够把控制者、数据来源和能力边界讲清楚，才算学会这一章。

4.8.4 源码阅读索引

阅读顺序	源码位置	读完应能回答
1	app/schemas/tool.py	参数怎样校验、默认值是什么
2	app/tools/get_document.py	found 与 not_found 如何返回
3	app/rag/document_store.py	索引怎样构建与缓存
4	app/tools/search_knowledge_base.py	查询准备与召回怎样连接
5	app/agents/circuit_agent.py	工具怎样注册、输入输出怎样包装
6	scripts/run_agent.py	独立 Agent 从哪里启动
7	app/api/chat.py	为什么 Web 走另一套主流程

进一步查阅官方文档：[Agents](#)、[Tools](#)。框架行为可能随版本变化，实际学习以当前依赖与本地代码为准。